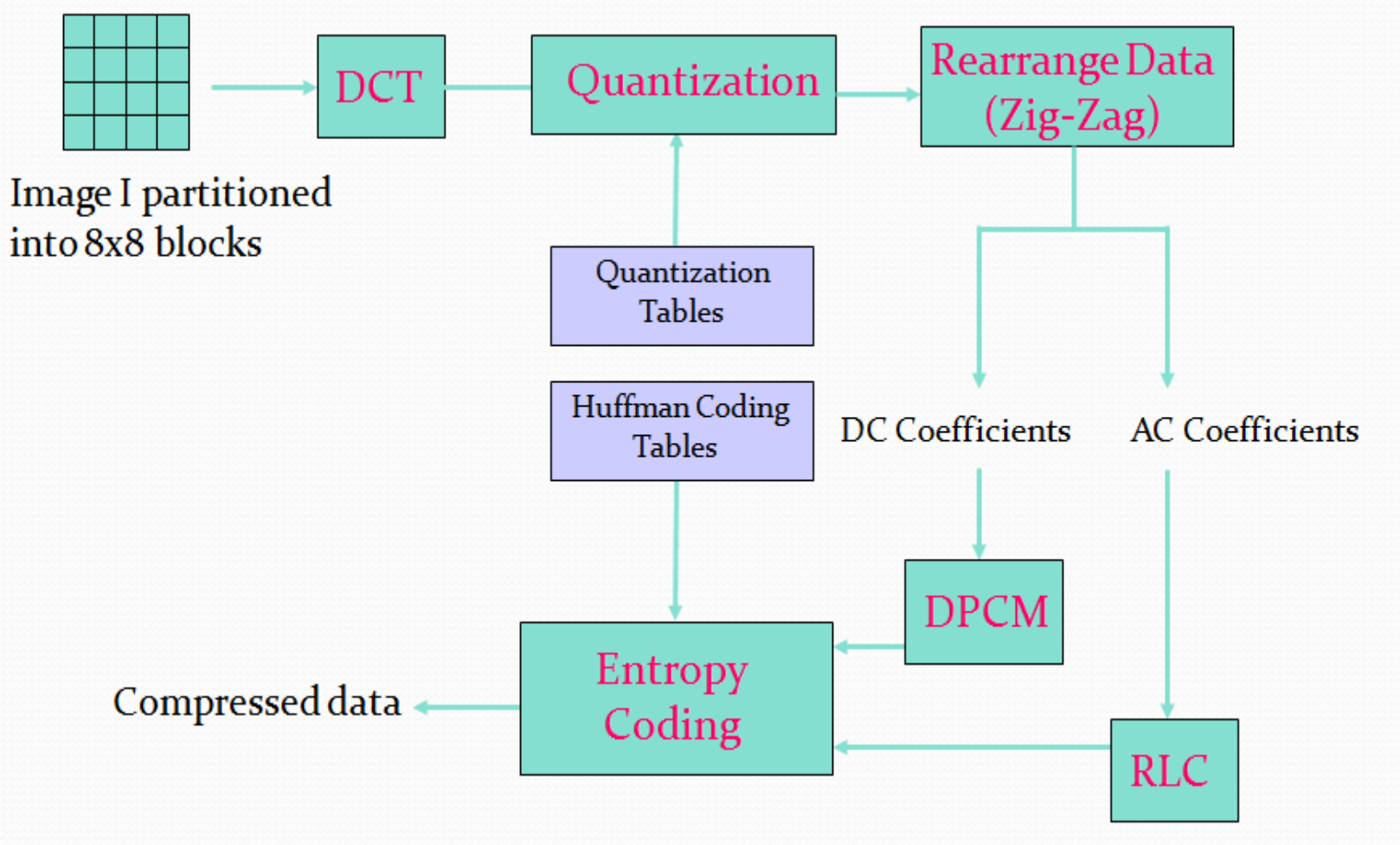
Πανεπιστήμιο Κύπρου
Τμήμα Πληροφορικής

ΕΠΛ445:
Ψηφιακή Επεξεργασία Εικόνας

Εργαστήριο 5

Βασικά Διάγραμμα Ροής Κωδικοποίησης JPEG



Βήμα 1: Μπλοκ μεγέθους 8x8 and DCT

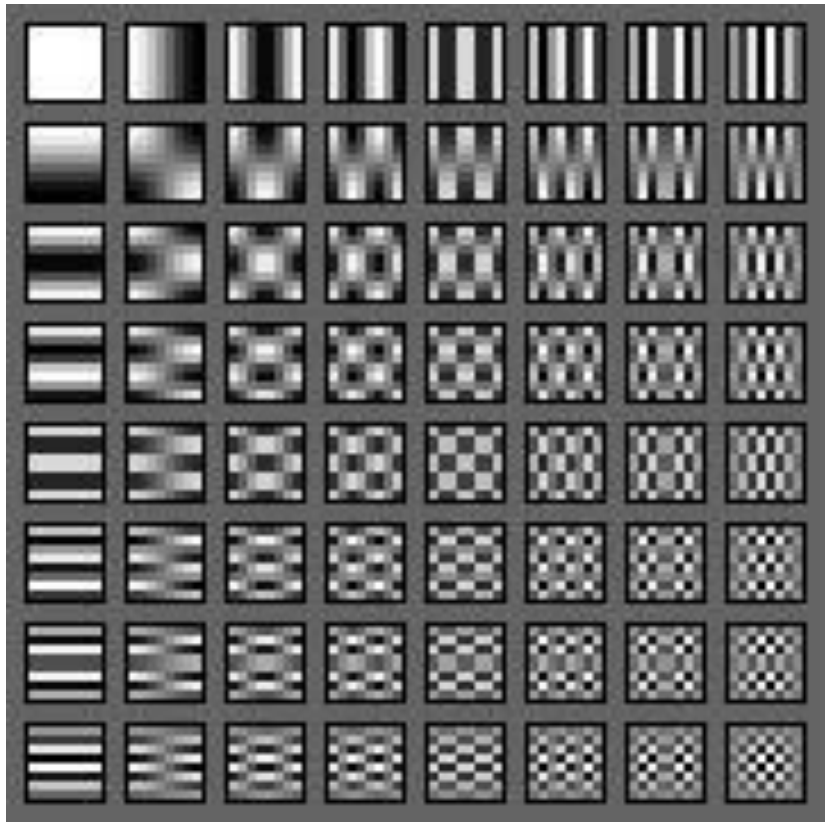
- Χωρίζουμε την εικόνα σε μικρά μπλοκ μεγέθους 8×8 pixels και μετασχηματίζουμε **κάθε μπλοκ** με DCT (Discrete Cosine Transform). Καθορίζουμε αυτά τα μπλοκ με $\tilde{\tilde{I}}_k(u, v)$

- <u>Κώδικας Python:</u>

```
vector_blocks = []
iHeight, iWidth = img.shape
# Image partitioned into 8x8 blocks
for startY in range(0, iHeight, 8):
   for startX in range(0, iWidth, 8):
         block = img[startY:startY + 8, startX:startX + 8]
         vector_blocks.append(block)
# DCT
block_f = np.float32(block)
block_dct = cv2.dct(block_f)
```

Σχόλια για τον μετασχηματισμό DCT

- Οι βασικές συναρτήσεις του DCT παρουσιάζονται ως εικόνες 8×8.
  - Οι οριζόντιες συχνότητες αυξάνονται από αριστερά προς τα δεξιά, ενώ οι κάθετες συχνότητες αυξάνονται από πάνω προς τα κάτω.
  - Η σταθερή τιμή της βασικής συνάρτησης που βρίσκεται στην πάνω αριστερή γωνία ονομάζεται DC βασική συνάρτηση (DC basis function).



Βήμα 2: Κβαντοποίηση συντελεστών του DCT

- Κβατοποιούμε τους συντελεστές του DCT χρησιμοποιώντας την εξίσωση:

$$\underset{\sim k}{I}^{Q}(u, v) = \mathrm{int}\left[\frac{\underset{\sim k}{I}(u, v)}{Q(u, v)} + 0.5\right]$$

- Διαιρούμε κάθε στοιχείο του μπλοκ με το αντίστοιχο στοιχείο ενός πίνακα κβαντοποίησης (κανονικοποίησης) 8x8 Q(u,v) ο οποίος θα δίνεται.
- Το αποτέλεσμα είναι ένας πίνακας ακέραιων τιμών, ο οποίος περιέχει πολλές μηδενικές τιμές.
- <u>Κώδικας Python:</u>

```
# Quantization of the DCT coefficients
q_table_factor = np.multiply(q_table, factor)
block_q = np.floor(np.divide(block_dct, q_table_factor) + 0.5)
```

Παράδειγμα Κβαντοποίησης

- Ένα DCT (Discrete cosine transformed) μπλοκ

$$\tilde{\tilde{\mathbf{I}}}_k = \begin{bmatrix} 1260 & -1 & -12 & -5 & 2 & -2 & -3 & 1 \\ -23 & -17 & -6 & -3 & -3 & 0 & 0 & -1 \\ -11 & -9 & -2 & 2 & 0 & -1 & -1 & 0 \\ -7 & -2 & 0 & 1 & 1 & 0 & 0 & 0 \\ -1 & -1 & 1 & 2 & 0 & -1 & 1 & 1 \\ 2 & 0 & 2 & 0 & -1 & 1 & 1 & -1 \\ -1 & 0 & 0 & -1 & 0 & 2 & 1 & -1 \\ -3 & 2 & -4 & -2 & 2 & 1 & -1 & 0 \end{bmatrix}$$

- Τυπικός πίνακας κβαντοποίησης (κανονικοποίησης):

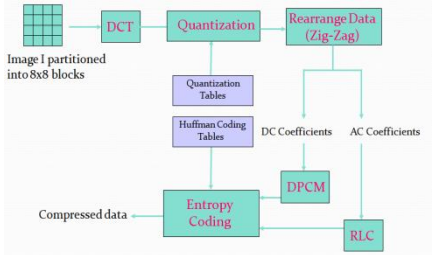
$$\mathbf{Q} = \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}$$

Παράδειγμα Κβαντοποίησης

- Ο κβατοποιημένος πίνακας DCT που προκύπτει

$$\tilde{\tilde{\mathbf{I}}}_{\mathrm{k}}^{\mathrm{Q}} = \begin{bmatrix} 79 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ -2 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

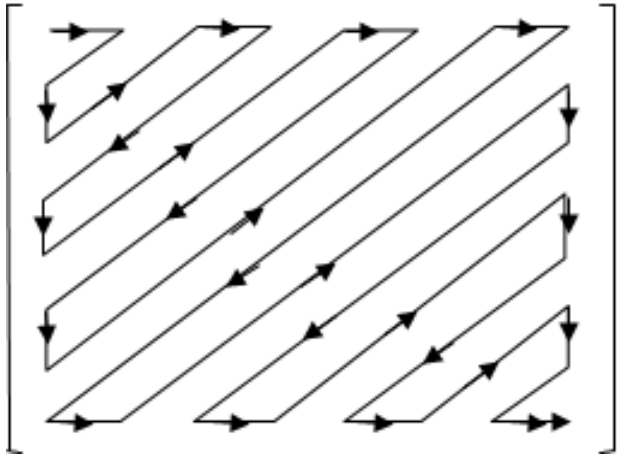
- Παρατηρείστε όλα τα μηδενικά. Η DC τιμή $\tilde{\tilde{\mathbf{I}}}_{\mathrm{k}}^{\mathrm{Q}}$ (0,0) βρίσκεται στην πάνω αριστερή γωνιά.
  - Η τιμή αυτή αποτελεί ένα μέτρο της **μέσης τιμής των 64 pixels** του μπλοκ της εικόνας.



Βήμα 3: Αναδιάταξη δεδομένων (Zig-Zag)

- Αναδιατάσσουμε το μπλοκ με τις κβαντοποιημένες τιμές ένα μονοδιάστατο διάνυσμα 64 στοιχείων.
- Στόχος είναι η ομαδοποίηση των συντελεστών χαμηλής συχνότητας στην αρχή του διανύσματος και των συντελεστών υψηλής συχνότητας στο τέλος.

$$\begin{bmatrix} 0 & 1 & 5 & 6 & 14 & 15 & 27 & 28 \\ 2 & 4 & 7 & 13 & 16 & 26 & 29 & 42 \\ 3 & 8 & 12 & 17 & 25 & 39 & 41 & 43 \\ 9 & 11 & 18 & 24 & 31 & 40 & 44 & 53 \\ 10 & 19 & 23 & 32 & 39 & 45 & 52 & 54 \\ 20 & 22 & 33 & 38 & 46 & 51 & 55 & 60 \\ 21 & 34 & 37 & 47 & 50 & 56 & 59 & 61 \\ 35 & 36 & 48 & 49 & 57 & 58 & 62 & 63 \end{bmatrix}$$



- Από το προηγούμενο παράδειγμα, το μονοδιάστατο διάνυσμα που προκύπτει είναι:

[79 0 -2 -1 -1 -1 0 0 -1 (55 ‘0’)]

Βήμα 4: DPCM για τους συντελεστές DC

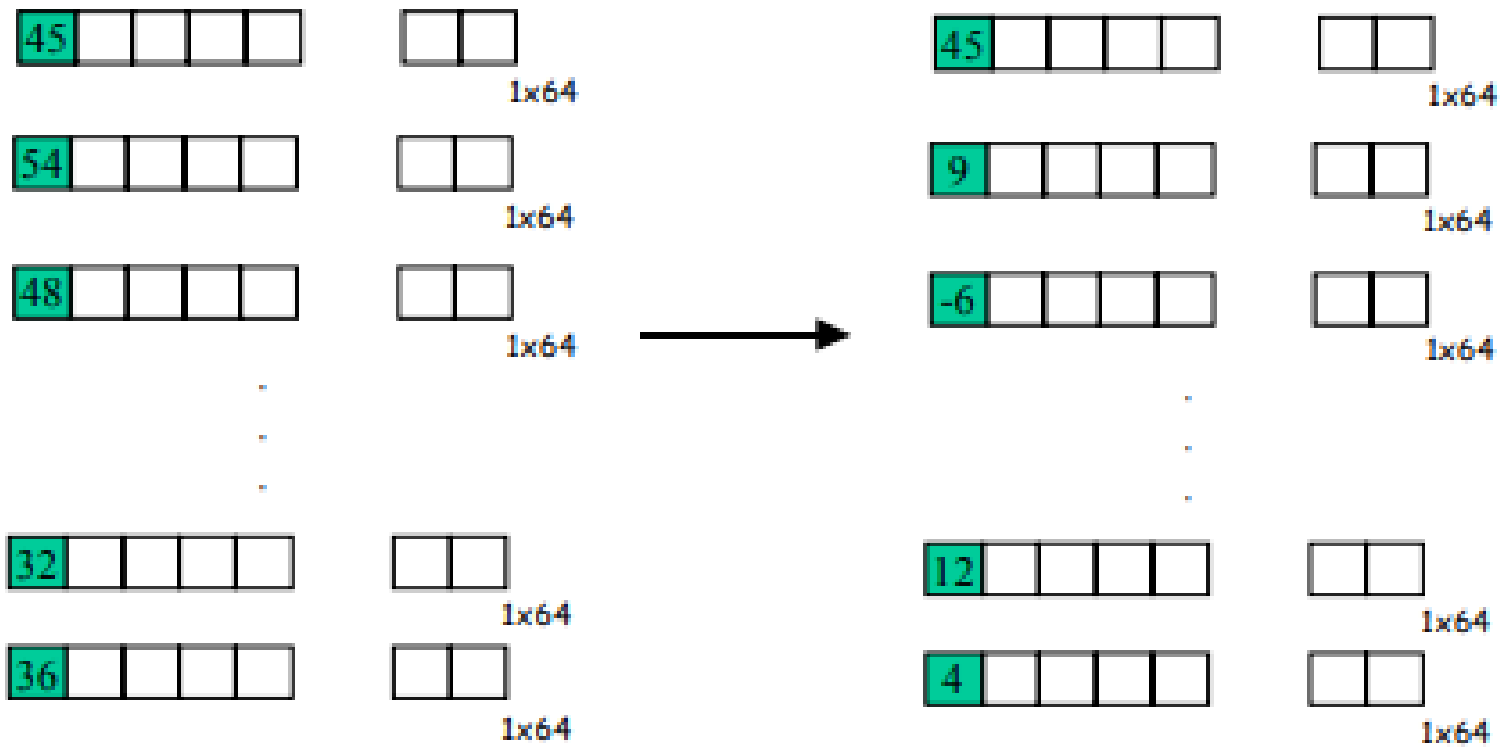
- Εφαρμόζουμε DPCM (Differential Pulse Code Modulation) στους DC συντελεστές μεταξύ γειτονικών μπλοκ, ώστε να μειώσουμε την εντροπία των DC τιμών, δηλαδή τον αριθμό των bits ανά pixel που απαιτούνται για την κωδικοποίηση της εικόνας.
- Υπολογίζουμε τη διαφορά μεταξύ της DC τιμής του τρέχοντος μπλοκ και της DC τιμής του προηγούμενου μπλοκ (αριστερά):

$$e(k) = \tilde{\tilde{I}}_k^Q(0, 0) - \tilde{\tilde{I}}_{k-1}^Q(0, 0)$$

- <u>Κώδικας Python:</u>

```
e = []
# leave first value of first vector as it is
e.append(vectors_zigzag[0][0])
for k in range(1, len(vectors_zigzag)):
    e.append(vectors_zigzag[k][0] - vectors_zigzag[k - 1][0])
```

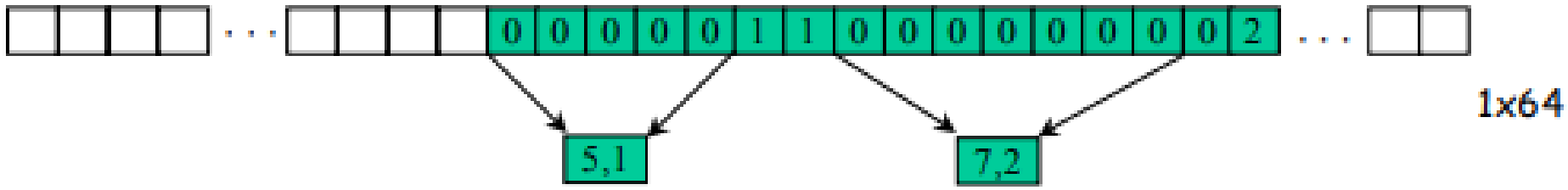
Παράδειγμα DPCM



- Αφήνουμε την πρώτη DC τιμή όπως είναι.
- Στη συνέχεια αφαιρούμε την DC τιμή του προηγούμενου μπλοκ από την DC τιμή του τρέχοντος μπλοκ.

Βήμα 5: RLC για τους συντελεστές AC

- Οι AC συντελεστές περιέχουν πολλές μηδενικές τιμές. Για να τους κωδικοποιήσουμε χρησιμοποιούμε ζεύγη (Skip, Value) όπου:
  - Skip είναι ο αριθμός των μηδενικών πριν από μια μη μηδενική τιμή.
  - Value είναι η μη μηδενική τιμή που ακολουθεί.
- Στο τέλος του διανύσματος προσθέτουμε το (0,0) για να δηλώσουμε το τέλος του μπλοκ (end-of-block).
- Παράδειγμα RLC (Run Length Coding).



Βήμα 6: Κωδικοποίηση Huffman

- Κωδικοποίηση Huffman των διαφορών DC
- Κωδικοποίηση Huffman των AC τιμών

Αλγόριθμος Huffman

- Από το σύνολο των επιπέδων γκρι (k) στο σύνολο των κωδικών λέξεων (c(k)).

- Βήματα του αλγόριθμου

Δημιουργούμε ένα **δυαδικό δέντρο (binary tree)** όπου τα κλαδιά επισημαίνονται με τα επίπεδα γκρι k_m και τις πιθανότητές τους $p_I(k_m)$:

(0) Διαγράφουμε τα k_m για τα οποία $p_I(k_m) = 0$.

(1) Βρίσκουμε τις **δύο μικρότερες πιθανότητες** $p_m = p_I(k_m)$, $p_n = p_I(k_n)$

(2) Τις αντικαθιστούμε με $p_{mn} = p_m + p_n$ για να δημιουργήσουμε ένα **νέο κόμβο** και μειώνουμε τη λίστα κατά 1 στοιχείο.

(3) Ονομάζουμε το κλαδί k_m με '1' και το k_n με '0'.

(4) Επιστρέφουμε στο βήμα (1), μέχρι να μείνει ένα στοιχείο στη λίστα (η ρίζα του δέντρου).

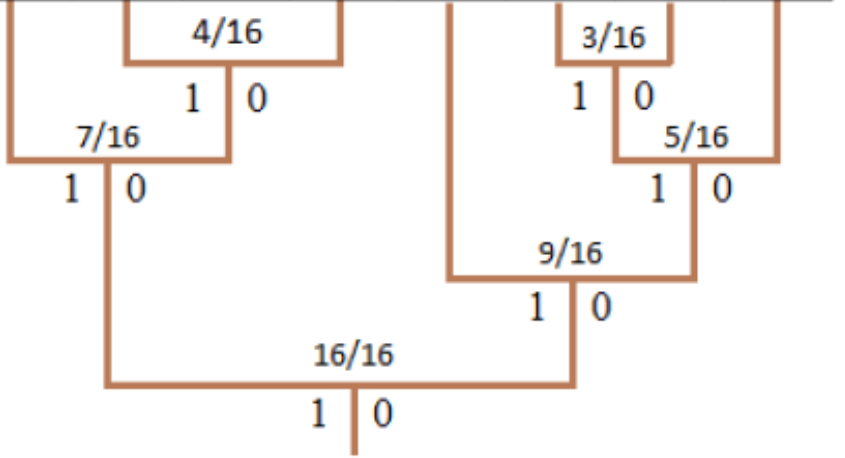
Στο βήμα (3), οι τιμές '1' and '0' ανατίθενται στα ζεύγη στοιχείων (k_m,k_n), καθώς δημιουργείται το δέντρο.

Παράδειγμα εφαρμογής του Huffman

I_2=

3	1	0	7
3	4	4	0
4	6	6	7
1	5	4	0

k	0	1	2	3	4	5	6	7
$p_I(k)$	3/16	2/16	0	2/16	4/16	1/16	2/16	2/16



Γράμμα που χρησιμοποιείται πιο συχνά => Μικρότερο μήκος επειδή είναι σημαντική πληροφορία

Διαβάζουμε ξεκινώντας από κάτω προς τα πάνω

c(k)	11	101
L[c(k)]	2	3

100	01	0011	0010	000
3	2	4	4	3

Μήκος λέξης

Κωδικοποίηση/Αποκωδικοποίηση Huffman στο OpenCV

- Πρώτα φορτώνουμε το Huffman3.py στο Session Storage

epl445_lab5a.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Files

..
sample_data
Huffman3.py

+ Code + Text

```
from Huffman3 import huffman, encode, decode, makenodes, iterate
import collections
import numpy as np

myList = [1, 2, 3, 4, 3, 2, 2, 1, 1, 1]
print("myList: ", myList)

# Find frequency of appearance for each value of the list
counter = collections.Counter(myList)

# Define list of probabilities as list of pairs (Unique item, Corresponding frequency)
probs = []

# Initialization of probabilities' list
for key, value in counter.items():
    print(key, value)
    probs.append((key, np.float32(value)))

# Creates a list of nodes ready for the Huffman algorithm 'iterate'.
symbols = makenodes(probs)

# Runs the Huffman algorithm on a list of "nodes". It returns a pointer to the root of a new tree of "internal nodes".
```

Κωδικοποίηση/Αποκωδικοποίηση Huffman στο OpenCV

```
from Huffman3 import huffman, encode, decode, makenodes, iterate
import collections
import numpy as np

myList = [1, 2, 3, 4, 3, 2, 2, 1, 1, 1]
print("myList: ", myList)

# Find frequency of appearance for each value of the list
counter = collections.Counter(myList)

# Define list of probabilities as list of pairs (Unique item, Corresponding
frequency)
probs = []
```

Κωδικοποίηση/Αποκωδικοποίηση Huffman στο OpenCV

```
# Initialization of probabilities' list
for key, value in counter.items():
    print(key, value)
    probs.append((key, np.float32(value)))
```

Key	Value
1	4
2	3
3	2
4	1

```
# Creates a list of nodes ready for the Huffman algorithm 'iterate'
symbols = makenodes(probs)

# Runs the Huffman algorithm on a list of "nodes". It returns a pointer to
the root of a new tree of "internal nodes"
root = iterate(symbols)
```

Κωδικοποίηση/Αποκωδικοποίηση Huffman στο OpenCV

```
# Encodes a list of source symbols
s = encode(myList, symbols)
print("Huffman encoding of myList: ", s)

# Decodes a binary string using the Huffman tree accessed via root
d = decode(s, root)
print("Huffman decoding of myList: ", d)
```

```
Huffman encoding of myList:  00000101101001001000000000
Huffman decoding of myList:  ['1', '2', '3', '4', '3', '2', '2', '1', '1', '1']
```

JPEG Decoder

- A. Αποκωδικοποίηση Huffman
  - Αποκωδικοποιούμε τους **DC συντελεστές** χρησιμοποιώντας Huffman.
  - Αποκωδικοποιούμε τους **AC συντελεστές** χρησιμοποιώντας Huffman.

- B. Αντίστροφο DPCM
  - <u>Κώδικας Python:</u>

```
# leave first value of first vector as it is
ie = []
ie.append(list_decode_dpcm[0])
    for k in range(1, len(list_decode_dpcm)):
        ie.append(list_decode_dpcm[k] + ie[k - 1])
```

JPEG Decoder

- C. Αντίστροφο RLC
- D. Αντίστροφο Zig-Zag
- E. Αντίστροφη κβαντοποίηση

$$\tilde{\tilde{\mathbf{I}}}_k^{lossy} = \mathbf{Q} \otimes \tilde{\tilde{\mathbf{I}}}_k^{Q}$$

- F. Αντίστροφο DCT

$$\mathbf{I}_k^{lossy} = \mathrm{IDCT}\left[\tilde{\tilde{\mathbf{I}}}_k^{lossy}\right]$$

- G. Επιστροφή στην εικόνα

Κώδικας Python:

```
# Inverse Quantization of the DCT coefficients
q_table_factor = np.multiply(q_table, factor)
np.multiply(block, q_table_factor)

# Inverse DCT
cv2.idct(block)
```

```
np.place(comp_image, comp_image>255.0, 255.0)      # saturation
np.place(comp_image, comp_image<0.0, 0.0)          # grounding
comp_image = np.uint8(comp_image)
```

SSIM στο OpenCV

- **SSIM (Structural Similarity Index)** είναι ένας δείκτης που εκφράζει την ποιότητα μιας εικόνας μετά τη συμπίεση. Συγκρίνει την αρχική εικόνα με την τελική εικόνα μετά τη συμπίεση.

```
import cv2
from skimage.metrics import structural_similarity as ssim

img = cv2.imread('cameraman.tif', 0)

ssim_value = ssim(img, img, data_range= img.max() - img.min())
print (ssim_value)
```

```
ssim_value = ssim(img.astype(np.float), comp_image.astype(np.float),
data_range= comp_image.max() - comp_image.min())
```